

Parcial III: Diseño, Adversidad y Robustecimiento de Protocolos de Comunicación

Materia: Comunicaciones II

Modalidad: Equipos de tres (3) estudiantes

1. Objetivo General

Diseñar e implementar un protocolo de comunicación de capa de aplicación cliente-servidor, evaluar su comportamiento ante interferencias y fallas simuladas en el canal, y reformular el protocolo mediante mecanismos de tolerancia a fallos.

2. Distribución de Roles en el Equipo

- **Estudiante A — Desarrollador del Servidor:**
 - Implementará la aplicación Servidor en un lenguaje de programación a su elección (ej. Python, Go, C++, C#).
 - Gestionará la concurrencia, el procesamiento de peticiones y respuestas estructuradas según el protocolo acordado.
- **Estudiante B — Desarrollador del Cliente:**
 - Implementará la aplicación Cliente en un lenguaje de programación **diferente** al utilizado por el Estudiante A (ej. si el Servidor es en Python, el Cliente puede ser en Java, JavaScript/Node.js, Rust, etc.).
 - Construirá la interfaz (CLI o GUI) para el envío/recepción de mensajes y serialización de datos.
- **Estudiante C — Agente de Adversidad (Atacante):**
 - Encargado de inyectar fallas en el canal de comunicación utilizando herramientas existentes (ej. Wireshark, Ettercap, Netem, Scapy, Clumsy, Proxy MITM) o mediante un script intermediario personalizado.
 - Documentará las vulnerabilidades detectadas en el protocolo inicial.

Fases de la Práctica

Fase 1: Diseño e Implementación Inicial del Protocolo: Definición de estructura y comunicación básica. **Diseño de la Tramas/Mensajes:** Los estudiantes A y B deben acordar la estructura del protocolo (cabeceras, delimitadores, longitud, tipo de mensaje, payload).

1. **Desarrollo:**
 - Estudiante A crea el Servidor en el **Lenguaje X**.
 - Estudiante B crea el Cliente en el **Lenguaje Y**.
2. **Prueba Inicial:** Establecer una comunicación fluida de envío y recepción de mensajes en un entorno controlado (Red Local o Loopback).

Fase 2: Inyección de Fallas y Análisis de Debilidades: Evaluación del comportamiento ante anomalías. El Estudiante C interceptará o alterará el canal de comunicación aplicando al menos **tres (3)** de las siguientes condiciones de falla:

- **Pérdida de Paquetes:** Descarte selectivo o aleatorio de datagramas/tramas.
- **Corrupción de Datos:** Alteración de bits/caracteres en el cuerpo del mensaje o cabecera.
- **Latencia y Desorden:** Retardo artificial o alteración del orden de llegada de los mensajes.
- **Reorganización/Inyección:** Duplicación de mensajes o inyección de tramas corruptas (Ataque Man-in-the-Middle o Replay).

Los Estudiantes A y B deberán registrar cómo reaccionan las aplicaciones (cuelgues, datos corruptos, desconexiones inesperadas).

Fase 3: Rediseño y Robustecimiento del Protocolo: Implementación de tolerancia a fallos.

Con base en los hallazgos de la Fase 2, el equipo completo modificará el protocolo agregando mecanismos como:

- **Verificación de Integridad:** Sumas de comprobación (Checksum, CRC32, hashes).
- **Control de Flujo y Errores:** Confirmaciones de recepción (ACK/NACK), temporizadores (Timeouts) y retransmisión (Stop-and-Wait o Go-Back-N).
- **Manejo de Excepciones:** Respuestas de error estandarizadas ante tramas malformadas.

Fase 4: Validación Final: Repetir las pruebas de interferencia de la Fase 2 demostrando que las aplicaciones mantienen la comunicación correcta, recuperan paquetes perdidos y descartan datos corruptos sin colapsar.

Entregables

1. **Código Fuente:**
 - Repositorio Git con el código del Servidor, Cliente y herramientas/scripts de fallas utilizados por el Agente de Adversidad.
 - Archivo README.md con instrucciones claras de ejecución y requisitos de entorno.
2. **Informe Técnico (PDF):**
 - **Especificación del Protocolo:**
 - **Bitácora de Adversidad:** Detalle de las herramientas usadas por el Estudiante C, ataques/fallas aplicados y capturas de tráfico (ej. Wireshark).
 - **Matriz de Errores y Soluciones:** Tabla comparativa indicando la falla inyectada, el comportamiento inicial de las aplicaciones y la solución implementada en el protocolo.